

PROJECT AND TEAM INFORMATION

Project Title

ShrinkByte: Lossless File Compression and Decompression Using Huffman Coding

Student / Team Information

Team Name: Team #:	ShrinkByte DSCPP-III-2026-T220
Team member 1 (Team Lead)	Agarwal, Deepanshi – 2510379831 2510379831@geu.ac.in 
Team member 2	Grover, Avika – 2510370032 2510370032@geu.ac.in 
Team member 3	Bhardwaj, Darsh – 2510370443 2510370443@geu.ac.in 
Team member 4	Saini, Aayushe – 2510378143 2510378143@geu.ac.in 

PROPOSAL DESCRIPTION

Motivation

Digital files occupy significant storage space and require bandwidth during transfer. Since too many files use and include repetitive information, but here this information can be represented more effectively without changing the real content.

Shrinkbyte here intends to create a file compression and decompression system, without any losses by using Huffman coding algorithm. This project shows us how data structures like frequency tables, minimum heaps and binary trees can be employed in order to reduce the size of files without any losses in original content of file.

This system here will enable the users to compress files for more efficient storage and then decompress them back, so that the raw data can be recovered without any alternations.

This project includes a real world implementation of data structures in C and object-oriented programming in C++, linking the theoretical concepts with the practical implementation.

State of the Art / Current solution

File compression is widely used in formats such as ZIP and GZIP to reduce storage requirements and improve data transfer efficiency. Huffman Coding is a known lossless compression technique that gives shorter codes to symbols that appear often and longer codes to symbols that appear rarely. It is found that modern compression tools often combine Huffman Coding with algorithms to boost performance. ShrinkByte focuses on implementing the core Huffman Coding process while showing how Data Structures in C and Object-Oriented Programming, in C++ can be used in a way.

Project Goals and Milestones

ShrinkBytes main goal is to make a working lossless file compressor and decompressor by using Huffman Coding and at the time using Data Structures in C and Object-Oriented Programming in C++.

The main goals are to look at the file data and find out how often each byte appears, to build a Huffman tree by using a heap to make the Huffman codes to turn the file into a compressed version to save the information needed to get the original file back to make the original file again and to check that the data is correct.

The system will also be checked based on how much it compresses how much space is saved and how fast it works.

The first part was learning about lossless compression and Huffman Coding designing the system layout and doing the work, on the frequency table and minimum heap; the next part was building the Huffman tree making the codes compressing, putting bits together dealing with the information and doing the decompression.

The final part will be to test files and tricky situations to make sure the exact same file comes back to look at how well it works and to write the instructions.

Project Approach

ShrinkByte will use Huffman Coding as the primary lossless compression algorithm. The input file will be read in binary mode and the frequency of each byte will be calculated using a frequency table. The unique bytes and their frequencies will be inserted into a minimum heap. The two lowest-frequency nodes will repeatedly be combined to construct the Huffman tree. Tree traversal will then generate unique binary codes for the bytes.

The generated codes will be used to encode the file contents into a compressed bit stream. The encoded bits will be packed into bytes, while necessary metadata and padding information will be stored to support decompression.

During decompression, the stored information will be read, the Huffman tree will be reconstructed and the encoded bit stream will be decoded to regenerate the original file.

The core data structures and algorithmic operations will be implemented in C, including the frequency table, minimum heap and Huffman tree.

System Architecture (High Level Diagram)

ShrinkByte System Architecture

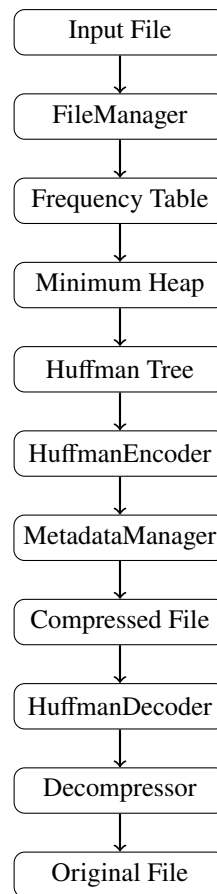


Diagram 1

The way the architecture is designed shows the process moving from the input file through frequency analysis, the construction of the Huffman tree, encoding, handling of the metadata and decompression in order to reconstruct the original file.

Project Outcome / Deliverables

The project will deliver a functional lossless file compression and decompression system capable of using Huffman Coding to reduce file size where effective compression is possible. The decompression module will reconstruct the original file without information loss.

The major deliverables will include a Huffman-based compressor and decompressor, C implementations of the required data structures, a C++ object-oriented application framework, compressed-file metadata handling, testing with different file types and edge cases, and performance evaluation using compression ratio, space saving and execution time.

The project will also include byte-level verification of the recovered file, source code, README/user documentation and the final project report.

Assumptions

- The input files will be restricted to the `.txt`, `.csv`, `.c` and `.cpp` formats.
- The input file will be valid, readable and accessible.
- The size of the input file expected will be under 1 GB.
- The files will be handled on a byte level.
- The file itself will not be altered during the process of compressing it.
- Accept for decompression only those files that are valid ShrinkByte compressed files.
- The file after it has been decompressed should be the same as the original file.

References

- [1] D. A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.
- [2] J. Ziv and A. Lempel, "A Universal Algorithm for Sequential Data Compression," *IEEE Transactions on Information Theory*, vol. 23, no. 3, pp. 337–343, 1977.
- [3] P. Deutsch, "DEFLATE Compressed Data Format Specification," RFC 1951, Internet Engineering Task Force, 1996.
- [4] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, MIT Press.
- [5] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed., Prentice Hall.
- [6] B. Stroustrup, *The C++ Programming Language*, 4th ed., Addison-Wesley.
- [7] E. S. Schwartz and B. Kallick, "Generating a Canonical Prefix Encoding," *Communications of the ACM*, vol. 7, no. 3, pp. 166–169, 1964, doi: 10.1145/363958.363991.
- [8] D. A. Lelewer and D. S. Hirschberg, "Data Compression," *ACM Computing Surveys*, vol. 19, no. 3, pp. 261–296, 1987.
- [9] A. Moffat and A. Turpin, "On the Implementation of Minimum-Redundancy Prefix Codes," *IEEE Transactions on Communications*, vol. 45, no. 10, pp. 1200–1207, 1997, doi: 10.1109/26.634683.